

Autocomplétion de recettes

Recherche intelligente avec React et TheMealDB API

Formation DWWM - La Plateforme_

Introduction

L'autocomplétion est un mécanisme essentiel sur le web moderne. Tu la retrouves partout : Google, YouTube, Amazon...

Dans ce projet, tu vas créer une application complète de recherche de recettes avec une autocomplétion professionnelle.

Qu'est-ce que l'autocomplétion ?

L'autocomplétion propose des suggestions à l'utilisateur pendant qu'il tape dans un champ de recherche.

Les suggestions s'adaptent en temps réel à chaque caractère saisi.

👉 **Important** : Autocomplétion = meilleure expérience utilisateur + recherche plus rapide + moins d'erreurs de frappe.

Le projet : recherche de recettes

Tu vas créer une application qui permet de :

- Rechercher des recettes de cuisine
- Voir des suggestions intelligentes pendant la frappe
- Afficher les résultats de recherche
- Consulter les détails d'une recette
- Naviguer au clavier dans les suggestions

Tu utiliseras l'API gratuite TheMealDB qui contient des milliers de recettes.

L'API TheMealDB

Qu'est-ce que TheMealDB ?

TheMealDB est une API gratuite qui fournit :

- Des milliers de recettes de cuisine
- Photos des plats
- Ingrédients et quantités
- Instructions de préparation
- Catégories (Beef, Chicken, Dessert...)
- Origines (British, Italian, Mexican...)

💡 **Astuce :** Aucune clé API nécessaire ! Tu peux l'utiliser directement.

Endpoints principaux

Rechercher par nom

<https://www.themealdb.com/api/json/v1/1/search.php?s={query}>

Exemple : `search.php?s=chicken`

Détails d'une recette

<https://www.themealdb.com/api/json/v1/1/lookup.php?i={id}>

Exemple : `lookup.php?i=52772`

Rechercher par première lettre

<https://www.themealdb.com/api/json/v1/1/search.php?f={letter}>

Exemple : `search.php?f=c`

Structure JSON retournée

L'API retourne un objet avec :

- meals : tableau des recettes trouvées
- idMeal : identifiant unique
- strMeal : nom de la recette
- strCategory : catégorie (Beef, Chicken...)
- strArea : origine (British, Italian...)
- strInstructions : étapes de préparation
- strMealThumb : URL de l'image
- strIngredient1 à strIngredient20 : ingrédients
- strMeasure1 à strMeasure20 : quantités

💡 **Astuce :** Les ingrédients et mesures vont de 1 à 20. Certains peuvent être vides.

Architecture de l'application

Les 3 pages

1. Page d'accueil (Home)

- Barre de recherche centrale
- Style moteur de recherche (Google-like)
- Autocomplétion active

2. Page de résultats (SearchResults)

- Grille de recettes correspondant à la recherche
- Cards avec photo, nom, catégorie
- Cliquables vers page détails

3. Page détails (RecipeDetail)

- Toutes les informations de la recette
- Photo grande taille
- Liste d'ingrédients
- Instructions complètes

Header global

Un header présent sur toutes les pages avec :

- Logo de l'application
- Barre de recherche avec autocomplétion
- Navigation vers l'accueil

L'autocomplétion intelligente

Principe de base

Au fur et à mesure que l'utilisateur tape, des suggestions apparaissent sous l'input.

Les suggestions proviennent de l'API TheMealDB.

Autocomplétion en 2 parties

👉 **Important** : Les suggestions sont divisées en 2 groupes distincts.

Partie 1 : Commence par...

Recettes dont le nom commence exactement par ce que l'utilisateur tape.

Exemple : si tu tapes 'chic', tu vois 'Chicken', 'Chick pea'...

Partie 2 : Contient...

Recettes qui contiennent la recherche mais ne commencent pas par.

Exemple : 'BBQ Chicken', 'Spicy Chicken Curry'...

Séparation visuelle

Un séparateur (ligne ou label) entre les 2 parties.

Si une partie est vide, pas de séparateur.

💡 **Astuce** : Cette division aide l'utilisateur à trouver rapidement ce qu'il cherche.

Algorithme de filtrage

```
const filterSuggestions = (meals, query) => {
  const lowerQuery = query.toLowerCase();

  // Partie 1 : commence par
  const startsWith = meals.filter(meal =>
    meal.strMeal.toLowerCase().startsWith(lowerQuery)
  );

  // Partie 2 : contient
  const contains = meals.filter(meal => {
    const name = meal.strMeal.toLowerCase();
    return name.includes(lowerQuery) && !name.startsWith(lowerQuery);
  });

  return { startsWith, contains };
};
```

Le debounce : optimiser les appels API

Problème sans debounce

Si tu appelles l'API à chaque frappe :

- Taper 'chicken' = 7 appels API !
- Ralentissement de l'app
- Consommation inutile de ressources
- API pourrait te bloquer

⚠ **Attention** : Sans debounce, ton app sera lente et gourmande en ressources.

Solution : le debounce

Le debounce attend un délai avant d'exécuter l'action.

Si l'utilisateur tape à nouveau avant la fin du délai, le timer recommence.

👉 **Important** : Attendre 300ms après la dernière frappe avant d'appeler l'API.

Implémentation du debounce

```
useEffect(() => {
  const timer = setTimeout(() => {
    if (searchTerm.length >= 2) {
      fetchSuggestions(searchTerm);
    }
  }, 300); // Attendre 300ms

  return () => clearTimeout(timer); // Cleanup
}, [searchTerm]);
```

Explication :

- setTimeout attend 300ms
- Si searchTerm change avant, clearTimeout annule le timer
- L'API n'est appelée qu'après 300ms sans frappe

💡 **Astuce :** Commence la recherche seulement si l'utilisateur a tapé au moins 2 caractères.

Navigation au clavier

Pourquoi ?

Les utilisateurs avancés utilisent le clavier pour naviguer plus vite.

Fonctionnalités à implémenter :

- Flèche ↓ : descendre dans les suggestions
- Flèche ↑ : remonter dans les suggestions
- Entrée : sélectionner la suggestion active
- Échap : fermer les suggestions
- Highlight : mettre en surbrillance la suggestion active

Gestion des touches

```
const handleKeyDown = (e) => {
  if (e.key === 'ArrowDown') {
    e.preventDefault();
    setSelectedIndex(prev =>
      Math.min(prev + 1, suggestions.length - 1)
    );
  } else if (e.key === 'ArrowUp') {
    e.preventDefault();
    setSelectedIndex(prev => Math.max(prev - 1, 0));
  } else if (e.key === 'Enter' && selectedIndex >= 0) {
    e.preventDefault();
    handleSelectSuggestion(suggestions[selectedIndex]);
  } else if (e.key === 'Escape') {
    setSuggestions([]);
  }
};
```

💡 **Astuce :** `e.preventDefault()` empêche le comportement par défaut (scroll de la page, soumission du formulaire).

React Router : navigation entre pages

Installation

```
npm install react-router-dom
```

Configuration

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Header />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/search" element={<SearchResults />} />
        <Route path="/recipe/:id" element={<RecipeDetail />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Navigation

Rediriger vers une page

```
import { useNavigate } from 'react-router-dom';

const navigate = useNavigate();
navigate('/search?q=' + query);
```

Récupérer paramètres URL

```
import { useSearchParams, useParams } from 'react-router-dom';

const [searchParams] = useSearchParams();
const query = searchParams.get('q');

const { id } = useParams();
```

Bonnes pratiques

Limiter le nombre de suggestions

Afficher maximum 10 suggestions pour ne pas surcharger l'interface.

```
startsWith: startsWith.slice(0, 5),
contains: contains.slice(0, 5)
```

Fermer suggestions au clic extérieur

Si l'utilisateur clique en dehors, fermer les suggestions.

```
useEffect(() => {
  const handleClickOutside = (e) => {
    if (!ref.current.contains(e.target)) {
      setSuggestions([]);
    }
  };
  document.addEventListener('click', handleClickOutside);
  return () => document.removeEventListener('click', handleClickOutside);
}, []);
```

Loader pendant le chargement

Afficher un indicateur de chargement pendant l'appel API.

```
const [loading, setLoading] = useState(false);

// Pendant fetch
setLoading(true);
// ...fetch...
setLoading(false);
```

Gérer les cas d'erreur

- Aucun résultat trouvé
- Erreur réseau
- API indisponible